

เอกสารประกอบมินิโปรเจกต์

ระบบจัดการคะแนนนักเรียน

สาธิตอัลกอริทึมการเรียงลำดับและการค้นหาข้อมูล

รายวิชา: โครงสร้างข้อมูลและขั้นตอนวิธี

ภาษาในการพัฒนา: Python 3 (Standard Library)

กำหนดส่งและนำเสนอ: วันที่ 5 ตุลาคม 2569 เวลา 13.00–16.00 น.

สมาชิกกลุ่ม

- รัตนพล ไชยศิริ — 684234034
- ธาดา ผ่องสุวรรณ — 684234018
- ทพีท พูลศิริสุข — 684234029
- จิรัชญา บุญเกิด — 684234001
- ชยางกูร สุวิญญู — 684234016
- ธนิษฐา สระทอง — 684234017

เอกสารฉบับนี้จัดทำขึ้นเพื่ออธิบายแนวคิด อัลกอริทึม โครงสร้างโปรแกรม วิธีใช้งาน และแนวทางการสาธิตโปรแกรมตามโจทย์มินิโปรเจกต์

1. บทนำและวัตถุประสงค์

มินิโปรเจกต์นี้พัฒนาโปรแกรมจัดการคะแนนนักเรียน โดยใช้ข้อมูลรหัสนักเรียน ชื่อ และคะแนนเป็นตัวอย่างในการสาธิตการทำงานของอัลกอริทึมการเรียงลำดับ (Sorting) และการค้นหาข้อมูล (Searching) ผู้ใช้สามารถเลือกอัลกอริทึม ดูผลลัพธ์ ดูสถิติการทำงาน และเปิดโหมด Trace เพื่อดูขั้นตอนระหว่างกระบวนการประมวลผลได้

วัตถุประสงค์

- ศึกษาและอธิบายหลักการของ Bubble Sort, Insertion Sort, Selection Sort และ Merge Sort
- ศึกษาและอธิบาย Sequential Search และ Binary Search
- พัฒนาโปรแกรม Python ที่แสดงผลการทำงานของอัลกอริทึมอย่างเป็นรูปธรรม
- เปรียบเทียบจำนวนการเปรียบเทียบ การสลับ/จัดวางข้อมูล และเวลาในการทำงาน
- สร้างสื่อประกอบการนำเสนอและทำให้สมาชิกทุกคนสามารถอธิบายโปรแกรมได้

2. ขอบเขตและความสามารถของระบบ

- เพิ่มข้อมูลนักเรียน ตรวจสอบรหัสซ้ำ และตรวจสอบคะแนนให้อยู่ในช่วง 0–100
- แสดงรายชื่อนักเรียนพร้อมคะแนนและเกรด
- ลบข้อมูลนักเรียนและโหลดข้อมูลตัวอย่าง 12 รายการ
- โหมดข้อมูลสังเคราะห์จำลองสำหรับการสาธิต (-data-test / --data-test / --mock-data) โหลดข้อมูลนักเรียน 35 รายการ
- เรียงข้อมูลตามคะแนน รหัสนักเรียน หรือชื่อ ทั้งแบบน้อยไปมากและมากไปน้อย
- ค้นหาด้วยรหัสหรือคะแนนด้วย Sequential Search และ Binary Search
- แสดงขั้นตอน Trace แบบจำกัดจำนวนบรรทัดเพื่อให้เหมาะกับการสาธิต
- แสดง Benchmark ของ Sorting ทั้ง 4 วิธี โดยไม่ทำให้โหมด Trace กระทบผล Benchmark

3. โครงสร้างโปรเจกต์

```
mini-project-sorting-searching/  
├── algorithms/ — sorting.py, searching.py  
├── models/ — student.py  
├── services/ — mock_data.py, score_manager.py  
├── tests/ — test_sorting.py, test_searching.py, test_score_manager.py, test_mock_data.py  
├── main.py — เมนูโปรแกรมหลัก (รองรับ -data-test, --data-test, --mock-data, --mock)  
└── README.md — คู่มือ ตารางความซับซ้อน และข้อชี้แจงจริยธรรมข้อมูล
```

การออกแบบแบ่งเป็นชั้น ได้แก่ โมเดลข้อมูล (Student) แกนอัลกอริทึม ฟังก์ชันบริการจัดการข้อมูลและข้อมูลจำลอง และส่วนติดต่อผู้ใช้แบบ Command Line ทำให้สามารถทดสอบและอธิบายแต่ละส่วนแยกกันได้

4. ความรู้พื้นฐาน: Sorting

Sorting คือการจัดเรียงข้อมูลตามเกณฑ์ที่กำหนด เช่น ค่ะแนจน้อยไปมาก หรือจากมากไปน้อย ประสิทธิภาพมักอธิบายด้วย Big-O ซึ่งใช้ออกแนวโน้มเวลาหรือหน่วยความจำเมื่อจำนวนข้อมูลเพิ่มขึ้น

อัลกอริทึม	หลักการทำงาน	Best / Average / Worst	จุดเด่น
Bubble Sort	เปรียบเทียบสมาชิกที่อยู่ติดกันและสลับเมื่ออยู่ผิดลำดับ	$O(n)$ / $O(n^2)$ / $O(n^2)$	เข้าใจง่าย และหยุดเร็วได้เมื่อข้อมูลเรียงแล้ว
Insertion Sort	หยิบสมาชิกทีละตัวแล้วแทรกในตำแหน่งที่ถูกต้อง	$O(n)$ / $O(n^2)$ / $O(n^2)$	เหมาะกับข้อมูลเกือบเรียงลำดับ
Selection Sort	เลือกค่าน้อยสุดหรือมากที่สุดจากส่วนที่เหลือมาวางตำแหน่งที่	$O(n^2)$ / $O(n^2)$ / $O(n^2)$	จำนวนการสลับไม่มาก
Merge Sort	แบ่งข้อมูลเป็นส่วนย่อย เรียงแต่ละส่วน แล้วผสานกลับ	$O(n \log n)$ / $O(n \log n)$ / $O(n \log n)$	ประสิทธิภาพสม่ำเสมอและเหมาะกับข้อมูลมาก

4.1 Bubble Sort

เริ่มจากสมาชิกคู่ติดกันทางซ้าย เปรียบเทียบกัน หากตัวซ้ายมากกว่าตัวขวาในกรณีเรียงน้อยไปมากให้สลับกัน เมื่อจบรอบค่าที่มากที่สุดจะถูกดันไปอยู่ด้านขวาสุด จากนั้นลดขอบเขตและทำซ้ำจนเรียงครบ หากไม่มีการสลับในรอบใดสามารถหยุดก่อนครบทุกประธานได้

4.2 Insertion Sort

มองส่วนซ้ายว่าเรียงแล้ว แล้วหยิบสมาชิกถัดไปมาเปรียบเทียบกับสมาชิกด้านซ้าย หากมากกว่าหรือผิดตำแหน่งให้เลื่อนสมาชิกเดิมไปทางขวาและแทรกสมาชิกที่หยิบมาในช่องว่าง วิธีนี้ทำงานดีเมื่อข้อมูลเกือบเรียงอยู่แล้ว

4.3 Selection Sort

ค้นหาสมาชิกที่มีค่าน้อยที่สุดในส่วนที่ยังไม่เรียง แล้วสลับมาไว้ตำแหน่งปัจจุบัน ทำซ้ำโดยเลื่อนตำแหน่งไปทางขวา จุดสังเกตคือแม้ข้อมูลจะเรียงอยู่แล้วยังต้องตรวจสอบหลายคู่ จึงมีเวลา $O(n^2)$ ทุกกรณี

4.4 Merge Sort

ใช้แนวคิด Divide and Conquer แบ่งรายการครึ่งหนึ่งซ้ำจนเหลือรายการขนาดหนึ่ง จากนั้นผสานรายการย่อยสองฝั่งโดยเลือกค่าที่เหมาะสมทีละตัว จึงมีความซับซ้อน $O(n \log n)$ แต่ใช้หน่วยความจำเสริม $O(n)$

5. ความรู้พื้นฐาน: Searching

อัลกอริทึม	หลักการทำงาน	ความซับซ้อน	เงื่อนไข
Sequential Search	ตรวจสอบจากต้นรายการไล่จนพบหรือหมดรายการ	Best $O(1)$, Average/Worst $O(n)$	ไม่ต้องเรียงข้อมูล
Binary Search	ตรวจสอบค่ากลาง แล้วตัดครึ่งช่วงค้นหาที่เป็นไปได้	Best $O(1)$, Average/Worst $O(\log n)$	เรียงข้อมูลก่อน

5.1 Sequential Search

เริ่มจากตำแหน่งแรก เปรียบเทียบค่าที่ค้นหา กับสมาชิกปัจจุบัน หากตรงกันให้บันทึกผล หากไม่ตรงให้ขยับไปตัวถัดไป วิธีนี้ใช้ได้กับข้อมูลที่เรียงไม่เรียง แต่ถ้าข้อมูลอยู่ท้ายรายการหรือไม่พบ จะต้องตรวจสอบหลายรายการ

5.2 Binary Search

กำหนดขอบเขตซ้ายและขวา แล้วคำนวณตำแหน่งกลาง เปรียบเทียบค่ากลางกับค่าที่ต้องการ หากค่าคั่นมากกว่าให้เลื่อนไปครึ่งขวา หากน้อยกว่าให้เลื่อนไปครึ่งซ้าย ทำซ้ำจนพบหรือขอบเขตหมด โปรแกรมจะตรวจสอบก่อนว่าข้อมูลเรียงแล้ว หากยังไม่เรียงจะเสนอให้เรียงด้วย Merge Sort ก่อน

ข้อควรระวัง: ห้ามใช้ Binary Search กับข้อมูลที่เรียงไม่เรียง เพราะผลลัพธ์อาจผิด แม้โค้ดจะทำงานโดยไม่มีข้อผิดพลาดก็ตาม

6. โหมด Trace และโหมดข้อมูลจำลองสำหรับการสาธิต

6.1 โหมด Trace

ในเมนูสถิติการเรียงลำดับ โปรแกรมจะถามว่า “ต้องการแสดงขั้นตอนการทำงานจริง (Trace) หรือไม่?” หากตอบ Y โปรแกรมจะบันทึกและแสดงขั้นตอนสำคัญ เช่น คู่อันดับที่กำลังเปรียบเทียบ การสลับค่า การแทรก การเลือกค่าต่ำสุด และการแบ่ง/ผสานของ Merge Sort ส่วน Search จะแสดงตำแหน่งหรือขอบเขตที่กำลังตรวจสอบ โหมด Trace ถูกแยกจาก Benchmark เพื่อให้สถิติการเปรียบเทียบยังสะท้อนการทำงานของอัลกอริทึมตามปกติ

6.2 โหมดข้อมูลสังเคราะห์จำลองสำหรับการสาธิต (Synthetic Mock Data Mode)

โปรแกรมรองรับการรันด้วยคำสั่ง `-data-test`, `--data-test` หรือ flag เดิม `--mock-data` / `--mock` เพื่อโหลดข้อมูลสังเคราะห์ขนาด 35 รายการ (รหัส TEST0001–TEST0035) แทนชุดข้อมูลตัวอย่างเดิม ทำให้เห็นความแตกต่างของจำนวน Comparisons และ Swaps ระหว่าง $O(n^2)$ เช่น Bubble Sort (~592 ครั้ง) กับ $O(n \log n)$ เช่น Merge Sort (~147 ครั้ง) ได้อย่างชัดเจนในการสาธิต

7. วิธีติดตั้งและใช้งาน

โปรเจกต์ใช้ Python Standard Library จึงไม่ต้องติดตั้งไลบรารีภายนอก แนะนำ Python 3.9 ขึ้นไป

คำสั่งเปิดโปรแกรม

```
cd /home/ryu/mini-project-sorting-searching
# โหมดปกติ (ข้อมูลตัวอย่าง 12 รายการ)
python3 main.py
# โหมดข้อมูลสังเคราะห์จำลองสำหรับการสาธิต (Mock Data 35 รายการ)
python3 main.py -data-test # หรือ --data-test, --mock-data, --mock
```

เมนูหลัก

เมนู	การทำงาน
1	แสดงรายชื่อนักเรียนทั้งหมด
2	เพิ่มข้อมูลนักเรียนและจำนวนเกรด
3	ลบข้อมูลด้วยรหัสนักเรียน
4	โหลดชุดข้อมูลตัวอย่าง / โหลดข้อมูลจำลองซ้ำ
5	สาธิต Sorting เลือกอัลกอริทึม เกณฑ์ ทิศทาง และ Trace
6	สาธิต Searching เลือก Sequential หรือ Binary และฟิลด์ค้นหา
7	Benchmark Sorting ทั้ง 4 วิธี
8	ล้างข้อมูลทั้งหมด
0	ออกจากโปรแกรม

ตัวอย่างลำดับการสาธิต Sorting

- เลือกเมนู 5
- เลือกอัลกอริทึม เช่น 1 สำหรับ Bubble Sort
- เลือกเกณฑ์ เช่น 1 สำหรับคะแนน
- เลือกทิศทาง เช่น 1 สำหรับน้อยไปมาก
- ตอบ Y เพื่อเปิด Trace
- ตอบ N หากไม่ต้องการบันทึกผลการเรียงเป็นลำดับหลัก
- อธิบายรายการ Trace แล้วชี้ให้เห็นผลลัพธ์และ Metrics

ตัวอย่างลำดับการสาธิต Searching

- เลือกเมนู 6
- เลือก Sequential Search หรือ Binary Search
- เลือกฟิลด์ เช่น รหัสนักเรียน
- กรอกรหัสที่ต้องการค้นหา
- สำหรับ Binary Search หากข้อมูลยังไม่เรียง ให้ตอบ Y เพื่อให้ระบบเรียงข้อมูลก่อน

- อธิบายตำแหน่งที่ตรวจสอบจาก Trace และผลลัพธ์ที่พบ

8. การทดสอบและผลการตรวจสอบ

โปรเจกต์มี Unit Tests ครอบคลุมโมเดลนักเรียน Sorting, Searching, ScoreManager, Mock Data และ CLI Flags ผลการตรวจสอบล่าสุดมีรายละเอียดดังนี้

คำสั่งทดสอบ

```
python3 -m unittest discover -s tests -v
pytest -v
```

ผลที่ตรวจสอบจริง: ผ่านทั้งหมด 63 รายการ (OK)

คำสั่งตรวจ Syntax

```
python3 -m py_compile main.py models/*.py algorithms/*.py services/*.py tests/*.py
```

ผลที่ตรวจสอบจริง: Exit Code 0 ไม่พบ Syntax Error หรือ Import Error

เกณฑ์ตรวจรับงาน

- อัลกอริทึมทั้ง 6 แบบให้ผลลัพธ์ถูกต้องตามข้อมูล
- Trace ถูกปิดเป็นค่าเริ่มต้นของฟังก์ชัน จึงไม่ทำลายการเรียกใช้เดิม
- เปิด Trace แล้วมีขั้นตอนที่อธิบายได้และจำกัดความยาว
- รองรับ -data-test, --data-test, --mock-data และ --mock สำหรับโหลดข้อมูลสังเคราะห์ 35 คนอย่างสมบูรณ์
- ข้อมูลสังเคราะห์ที่ไม่มีข้อมูลส่วนบุคคลจริงและใช้รหัส/ชื่อปลอมชัดเจน
- Binary Search ปฏิเสธหรือจัดการข้อมูลที่ยังไม่เรียงอย่างชัดเจน
- Benchmark ไม่ได้รับผลกระทบจาก Trace
- ชุดทดสอบอัตโนมัติ 63 รายการและการ Compile ผ่าน

9. แนวทางแบ่งการนำเสนอ 15 นาที

เวลา	เนื้อหา	ผู้รับผิดชอบแนะนำ
0:00–1:00	แนะนำหัวข้อและสมาชิก	สมาชิก 1
1:00–4:00	อธิบาย Sorting 4 แบบและ Big-O	สมาชิก 2–3
4:00–6:00	อธิบาย Sequential และ Binary Search	สมาชิก 4
6:00–11:00	สาธิตโปรแกรม Trace และ Mock Data Mode	สมาชิก 5
11:00–13:00	อธิบายโครงสร้างโค้ดและ Unit Tests	สมาชิก 6
13:00–15:00	สรุป เปรียบเทียบ และตอบคำถาม	ทุกคน

10. รายละเอียดเชิงเทคนิค

ส่วนนี้อธิบายการออกแบบภายในเพื่อให้ผู้พัฒนาหรือผู้ตรวจงานสามารถติดตามเส้นทางของข้อมูลตั้งแต่รับเข้าระบบจนถึงแสดงผลได้อย่างเป็นระบบ

10.1 โมเดลข้อมูลและการตรวจสอบ (Data Model & Validation)

ข้อมูลนักเรียนถูกแทนด้วย Student ซึ่งประกอบด้วยรหัสนักเรียน ชื่อ และคะแนน

โดยกำหนดเงื่อนไขว่ารหัสและชื่อต้องไม่เป็นค่าว่าง คะแนนต้องอยู่ระหว่าง 0 ถึง 100 และรหัสนักเรียนต้องไม่ซ้ำกันใน ScoreManager เมื่อสร้างหรือเพิ่มข้อมูล ระบบคำนวณเกรดจากคะแนนโดยอัตโนมัติ
จึงลดโอกาสที่ข้อมูลคะแนนและเกรดจะไม่สอดคล้องกัน

10.2 การไหลของข้อมูล (Data Flow)

ผู้ใช้เลือกคำสั่งจาก main.py → main.py แปลงคำสั่งเป็นพารามิเตอร์ → ScoreManager

ตรวจสอบสถานะและเลือกอัลกอริทึม → algorithms/sorting.py หรือ algorithms/searching.py ประมวลผล →
ส่งผลลัพธ์ Metrics และ Trace กลับมา → main.py แสดงตาราง ผลการค้นหา และสถิติบนหน้าจอ

10.3 Pseudocode ของขั้นตอนสำคัญ

Bubble Sort:

- 1) ทำซ้ำจากรอบแรกถึงรอบสุดท้าย
- 2) เปรียบเทียบสมาชิกคู่ติดกันในช่วงที่ยังไม่เรียง
- 3) สลับเมื่ออยู่ผิดลำดับ
- 4) หากไม่มีการสลับ ให้หยุดก่อนกำหนด

Binary Search:

- 1) กำหนด left = 0 และ right = n - 1
- 2) คำนวณ mid จากช่วงปัจจุบัน
- 3) ถ้าค่ากลางตรงกับค่าค้น ให้คืนผลลัพธ์
- 4) ถ้าค่าค้นมากกว่า ให้เลื่อน left ไปทางขวา มิฉะนั้นเลื่อน right ไปทางซ้าย
- 5) ทำซ้ำจน left มากกว่า right

10.4 ความถูกต้องและ Invariant

Bubble/Insertion/Selection Sort ต้องรักษาเงื่อนไขว่าส่วนหน้าที่ผ่านมาการประมวลผลแล้วอยู่ในลำดับที่ถูกต้อง ส่วน Merge Sort ต้องคืนผลจากการผสานสองลิสต์ย่อยที่เรียงแล้วให้เป็นลิสต์ที่เรียงแล้ว สำหรับ Binary Search เงื่อนไขสำคัญคือช่วง [left, right] ต้องครอบคลุมคำตอบที่ยังเป็นไปได้ และข้อมูลต้องเรียงตาม key ก่อนเริ่มค้นหา

10.5 การเก็บ Metrics และ Trace

SortMetrics และ SearchMetrics แยกจำนวน Comparisons จำนวน Swaps/Shifts/Merges ตำแหน่งที่พบ และเวลาประมวลผล โดยใช้ time.perf_counter() สำหรับจับเวลา Trace ถูกเปิดด้วยพารามิเตอร์ trace=True และมี max_trace_lines เพื่อควบคุมขนาดผลลัพธ์ เมื่อ trace=False ค่า Trace จะว่างและการทำ Benchmark ไม่ถูกบันทึกด้วยความสารถ

10.6 การจัดการข้อผิดพลาดและกรณีขอบ (Error & Edge Cases)

ระบบรองรับลิสต์ว่าง ข้อมูลหนึ่งรายการ ข้อมูลซ้ำ ข้อมูลเรียงแล้ว ข้อมูลเรียงย้อนกลับ ค้นหาไม่พบ รหัสซ้ำ คะแนนนอกช่วง และ Binary Search กับข้อมูลไม่เรียง โดยแจ้งข้อผิดพลาดหรือให้ผู้ใช้นัยนัยการเรียงข้อมูลก่อนดำเนินการต่อ

10.7 ความปลอดภัย จริยธรรมข้อมูล และการดูแลโค้ด (Data Ethics & Privacy)

โปรเจกต์ไม่มีการฝังรหัสผ่านหรือ API key ข้อมูลทั้งหมดที่ใช้งานเป็นข้อมูลสังเคราะห์ 100% (Synthetic Data) เช่น รหัส TEST0001–TEST0035 และชื่อ นักเรียนทดสอบ 001–035 ไม่มีการใช้ คัดลอก หรือเผยแพร่ข้อมูลส่วนบุคคล ชื่อจริง หรือรหัสนักศึกษาจริง การอ้างอิงเว็บไซต์ระบบบริการการศึกษา SKRU มีวัตถุประสงค์เพื่อศึกษาโครงสร้างและบริบทของกลุ่มเรียนเท่านั้น มิใช่การนำข้อมูลส่วนบุคคลจริงมาใช้

10.8 แนวทางต่อยอดทางเทคนิค

สามารถต่อยอดเป็น GUI หรือเว็บแอป เพิ่มการนำเข้า/ส่งออก CSV เพิ่มการวัดผลกับข้อมูลหลายขนาด ทำกราฟเปรียบเทียบเวลา เพิ่ม property-based testing และเพิ่มการบันทึกผลการทดลองเป็นไฟล์ เพื่อวิเคราะห์ผลกระทบของจำนวนข้อมูลต่อ Big-O ได้ชัดเจนขึ้น

11. สรุป

โปรแกรมนี้เชื่อมโยงทฤษฎีกับการทำงานจริง โดยใช้ระบบคะแนนนักเรียนเป็นบริบทที่เข้าใจง่าย ผู้ชมสามารถเห็นทั้งผลลัพธ์สุดท้าย สถิติการทำงาน และลำดับขั้นตอนผ่าน Trace จุดสำคัญในการนำเสนอคืออธิบายว่าแต่ละอัลกอริทึมเลือกตรวจหรือจัดข้อมูลอย่างไร เหตุใด Binary Search จึงต้องใช้ข้อมูลที่เรียงแล้ว และ Merge Sort จึงมีความซับซ้อนดีกว่าอัลกอริทึมแบบ $O(n^2)$ เมื่อข้อมูลมีจำนวนมาก

ไฟล์โปรแกรม: mini-project-sorting-searching.zip

โฟลเดอร์โปรเจกต์: /home/ryu/mini-project-sorting-searching